

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI

KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ THI KIỂM TRA (Thời gian: 60 phút)

MÔN: PHÁT TRIỂN ỨNG DỤNG PHÍA SERVER

Đề bài: Xây dựng ứng dụng ASP .NET Core WEB API có tên <Họ và tên><Mã số sinh viên> (vd: NguyenVanA1234) thỏa mãn các yêu cầu sau:

1. Yêu cầu chung

- **Tổ chức code:**
 - Coding convention sử dụng PascalCase với tên các trường, class, interface.
 - Sử dụng camelCase với các biến.
 - Sử dụng _camelCase với các trường có access modifier là private.
- **Cây thư mục code tuân thủ cấu trúc sau:**
 - **Properties:** các config chạy ứng dụng.
 - **Constants:** các hằng số.
 - **Controllers:** các API Controller.
 - **DbContexts:** các context entity framework.
 - **Dtos:** chia các thư mục con theo nghiệp vụ.
 - **Entities:** các entity map với bảng trong cơ sở dữ liệu.
 - **Exceptions:** các class xử lý ngoại lệ.
 - **Migrations:** các migration tạo các cấu trúc bảng quan hệ.
 - **Services:** xử lý các logic nghiệp vụ.
 - **Implements:** triển khai các interface.
 - **Interfaces:** khai báo các interface.
 - **Utils:** các hàm hữu ích dùng chung.
 - **appsetting.json:** các setting của ứng dụng dạng json.
 - **Program.cs.**

2. Yêu cầu hệ thống và kỹ thuật

- **Chuẩn hóa API Response:** Mọi API phải trả về dữ liệu dưới định dạng JSON thống nhất như sau:

```
{
  "isSuccess": true/false,
  "data": <dữ liệu trả về hoặc null>,
  "message": "<thông báo thành công hoặc lỗi>",
  "code": <mã code nghiệp vụ hoặc http code>
}
```

- **HTTP Status Code:** Luôn trả về HTTP Status **200 OK** ở mọi response, trừ các trường hợp ngoại lệ tuân theo chuẩn RESTful API (ví dụ: **400 Bad Request** cho lỗi validate, **404 Not Found** khi không tìm thấy tài nguyên, **500 Internal Server Error** cho lỗi hệ thống).
- **Xử lý DTO & Validation:**
 - Các class Dto phải xử lý trim đối với các trường kiểu dữ liệu string.
 - Validate model bằng các built-in annotation attribute với các class Dto phục vụ cho việc create, update, delete (bao gồm bắt buộc nhập, số lượng ký tự, ...).
- **Cơ sở dữ liệu & ORM:**
 - Kết nối đến cơ sở dữ liệu (SQL Server hoặc MySQL) sử dụng Entity Framework Core theo hướng Code First.
 - Các bảng phải có khoá chính (primary key) dạng số nguyên và tự tăng.
 - Các bảng có quan hệ khoá ngoại phải được tạo liên kết khoá ngoại trong migration.
- **Logic & Naming Convention:**
 - Các câu lệnh truy vấn cơ sở dữ liệu sử dụng các method Linq, keyword Linq.
 - Xử lý logic nghiệp vụ theo design pattern Dependency injection (DI) dưới dạng các class service được inject vào controller khi sử dụng.
 - Xử lý logic trả ra các ngoại lệ thông qua class **UserFriendlyException** kế thừa từ **Exception**.
 - Đặt tên các class theo dạng <Tên class><Mã số sinh viên><Đề thi số> (vd: **EmployeeDto1234De2**).

3. Bài toán cụ thể

Cho quan hệ n-n giữa **Nhân viên** (gồm: tên nhân viên, mã nhân viên (kiểu ký tự), email; yêu cầu mã nhân viên và email không được trùng) và **Dự án** (gồm: tên dự án, mã dự án; yêu cầu tên và mã dự án không được trùng). Trong bảng quan hệ (Phân công), thêm một trường lưu **Số giờ làm việc** (kiểu số nguyên) để quản lý thời gian nhân viên tham gia dự án.

Thực hiện các chức năng sau:

1. Tạo migration và update vào database cấu trúc các bảng của bài toán trên.
2. Tạo các API **thêm, sửa, xóa** Nhân viên (lưu ý kiểm tra trùng mã nhân viên và email khi thêm, sửa).
3. Tạo API xem **danh sách có phân trang** đối với Nhân viên. Yêu cầu hỗ trợ phân trang bằng **PageSize** và **PageIndex**, đồng thời cho phép lọc gần đúng theo **tên nhân viên** hoặc **mã nhân viên** thông qua trường **Keyword** truyền vào.
4. Tạo API liệt kê danh sách **những dự án** mà một nhân viên dành nhiều giờ làm việc nhất (sắp xếp giảm dần theo số giờ). Đầu vào là ID Nhân viên, đầu ra là danh sách Dự án gồm tên dự án, mã dự án và số giờ làm việc.

Ghi chú: Sinh viên đọc kỹ đề, cán bộ coi thi không giải thích gì thêm.

----- Kết thúc -----